



High Level Design Document

Global Mobility Analyzer Application

Revision Number: 2.0

Last Date of Revision: 02/06/2026

Submitted By:

Fidha Manaph

Chandrajyothi Parambi Biju

Document Version Control

Date Issued	Version	Description	Author
1/05/2026	1	Initial HLD	Fidha Manaph, Chandrajyothi PB
2/06/2026	2	Updated HLD	Fidha Manaph, Chandrajyothi PB

Contents

Abstract	i
1. Introduction	5
1.1 Why this is a High-Level Document	5
1.2 Scope	5
1.3 Definitions	5
2. General Description.....	6
2.1 Product Perspective	6
2.2 Problem Statement	6
2.3 Proposed Solution	6
2.4 Further Improvements	7
2.5 Data Requirements	7
2.6 Tools Used	7
2.7 Constraints	8
3. Design Details.....	8
3.1 Process Flow	8
3.1.1 Data Ingestion	8
3.1.2 Exploratory Data Analysis	8
3.1.3 Data Validation	9
3.1.4 Data Preprocessing	9
3.1.5 Feature Engineering	10
3.1.6 Data Transformation	11
3.1.7 Class Imbalance Handling	11
3.1.8 Model Training	12
3.1.9 Hyperparameter Optimization	12
3.1.10 Model Evaluation	13
3.1.11 Model Pusher	13
3.1.12 Prediction Pipeline	13
3.1.13 FastAPI Web Application	14
3.1.14 Deployment Process	15
3.2 Event Logs.....	15
3.3 Error Handling	15
4. Performance	16
5. Conclusion	16
6. References	16

Abstract

Assessing visa applications manually can be time-consuming, inconsistent, and difficult due to the large number of applicant and employer-related factors involved in immigration decisions. The Visa Approval Prediction System addresses this problem by using machine learning techniques to predict the likelihood of visa approval or denial based on historical immigration application data. The system helps automate initial case screening, reduce manual effort, and assist applicants and consultants in identifying potential risk factors affecting visa decisions.

1 Introduction

1.1 Why this is a High-Level Document

The purpose of this High-Level Design (HLD) Document is to provide an overall architectural view of the Visa Approval Prediction System and define the major components involved in the application. This document helps in understanding how different modules interact with each other and serves as a reference during development, deployment, and maintenance of the system.

The HLD will:

- Present the overall system architecture and workflow
- Describe the machine learning pipeline used for visa approval prediction
- Explain the user interface and FastAPI-based web application
- Describe the database, cloud, and deployment architecture
- Define the interaction between preprocessing, training, prediction, and analysis modules
- Describe the integration of AWS cloud services and Docker deployment
- Include important non-functional requirements such as:
 - Security
 - Reliability
 - Scalability
 - Maintainability
 - Portability
 - Performance

1.2 Scope

The High-Level Design (HLD) document outlines the overall architecture of the Visa Approval Prediction System. It covers the machine learning pipeline, application workflow, database structure, deployment setup, and cloud integration. It explains how data preprocessing, model training, prediction, and risk analysis modules interact, using clear technical language suitable for developers, administrators, and stakeholders.

1.3 Definitions

<i>Term</i>	<i>Description</i>
Database	Collection of all information monitored by the system
AWS	Amazon Web Services
IDE	Integrated Development Environment

2. General Description

2.1 Product Perspective

Global Mobility Analyzer Application is a Machine Learning application that predicts whether visa will be approved or denied using historical immigration application data

2.2 Problem Statement

The proposed solution addresses the challenges of manual visa assessment, which can be time-consuming, inconsistent, and prone to human error when handling large volumes of applications. By leveraging machine learning techniques, the system provides automated predictions that support faster preliminary evaluation of visa applications.

The key objectives of the system are:

- To analyze applicant and employer characteristics and predict the probability of visa approval or rejection.
- To identify important factors that influence visa decisions through data analysis and visualization.
- To reduce manual effort and improve consistency in the visa screening process through automated predictions.
- To provide real-time prediction capabilities through a web-based application.
- To support applicants, employers, and immigration consultants with data-driven decision-making and risk assessment.

2.3 Proposed Solution

We will use a machine learning-based approach to build the US Visa Approval prediction system. The process begins with loading data from the database (MongoDB), followed by data preprocessing where missing values are handled and feature engineering is performed to derive meaningful attributes such as company age. Exploratory Data Analysis (EDA) is carried out to understand data distributions and relationships between features.

Next, multiple classification algorithms are trained and evaluated using a standardized pipeline. Model performance is compared using evaluation metrics such as accuracy and F1-score to identify the best-performing model. Hyperparameter tuning is then applied to improve the performance of the selected models, with techniques such as GridSearchCV used to optimize results.

After evaluation, the best model is finalized based on performance metrics and compared against the existing production model. If the newly trained model shows improved performance, it is accepted and pushed to AWS S3 for deployment. The final system is then integrated with a FastAPI-based web application to enable real-time predictions, ensuring a scalable and production-ready solution.

2.4 Further Improvements

The system can be further improved by integrating advanced machine learning models to enhance prediction accuracy. Automated model retraining can be implemented to effectively handle data drift over time. Additionally, the system can be extended to support more visa categories and country-specific immigration policies.

2.5 Data Requirements

The Visa Approval Prediction System requires applicant and employer-related data to predict whether a visa application will be approved or denied. The required data includes applicant details such as continent, education level, work experience status, and job training requirement, along with employer information such as number of employees, year of establishment, region of employment, prevailing wage, wage unit, and full-time position status. The dataset also contains a unique case_id for each application and a case_status attribute, which represents the target variable used for

training and evaluating the machine learning models.

2.6 Tools Used



1. Python: Used for implementing the machine learning pipeline and backend logic of the system.
2. Scikit-learn: Used for preprocessing, training, tuning, and evaluating machine learning models.
3. PyMongo: Used for storing and retrieving visa application and prediction data from MongoDB.
4. Neuro ML: Used for automated hyperparameter tuning and model optimization.
5. Pandas: Used for data manipulation, preprocessing, and handling structured dataset operations efficiently.
6. Plotly: Used for creating interactive visualizations and analytical graphs during data analysis and reporting.
7. Evidently AI: Used for detecting and monitoring data drift by comparing incoming data distributions with the training dataset.
8. FastAPI: Used to build the prediction API and web application interface.
9. AWS: Used for cloud hosting, deployment, and managing the scalable infrastructure of the application.
10. MongoDB: Used for storing visa application records, prediction results, and system-related data in a NoSQL database.
11. Git: Used for version control and managing source code changes during development.
12. Visual Studio Code: Used as the development environment for coding, debugging, and project

2.7 Constraints

Cloud deployment and model training may require high computational resources for large-scale data processing. Predictions generated by the system are probabilistic and should not be considered official immigration decisions.

3. Design Details

3.1 Process Flow

The process flow of the Visa Approval Prediction System illustrates the sequence of steps involved in data preprocessing, machine learning model training, and prediction generation. The overall workflow of the system is shown in the process flow diagram below.



3.1.1 Data Ingestion

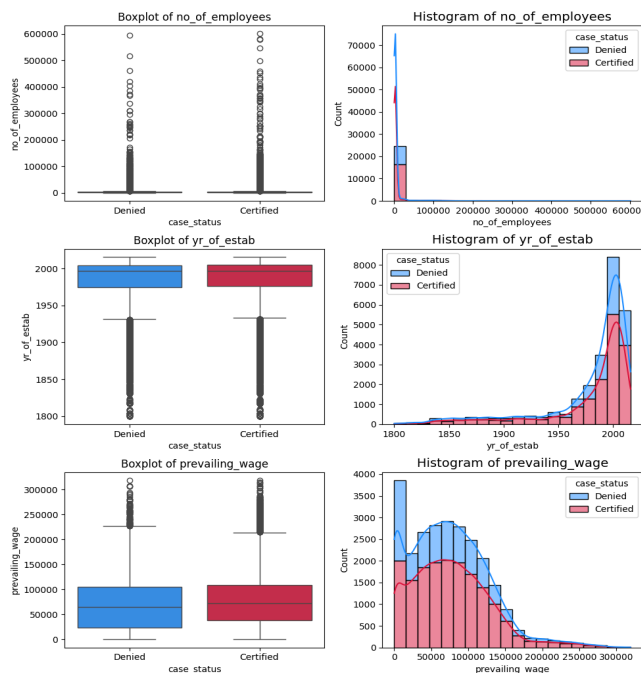
The Data Ingestion component is responsible for collecting visa application data from MongoDB and preparing it for further processing. The dataset is extracted from the database and stored locally in CSV format. The data is then divided into training and testing datasets to support machine learning model development and evaluation. This component serves as the entry point of the machine learning pipeline and ensures that data is consistently available for downstream processes.

3.1.2 Exploratory Data Analysis

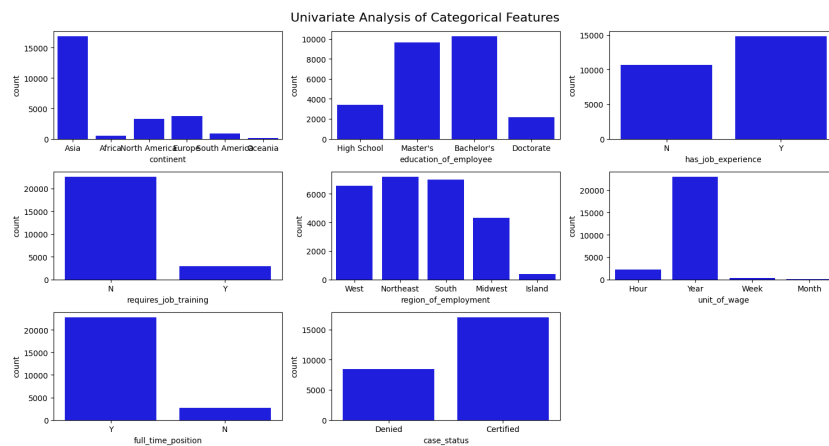
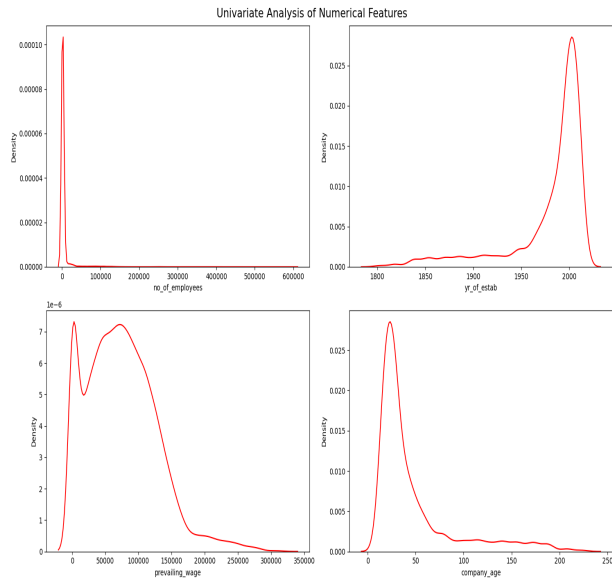
Exploratory Data Analysis (EDA) was performed to gain a comprehensive understanding of the visa application dataset before model development. Both univariate analysis and multivariate analysis were conducted to identify patterns, trends, and relationships within the data.

Univariate analysis was performed to examine the distribution and characteristics of individual features in the dataset. Various visualizations, including histograms, count plots, and box plots, were used to analyze variables such as education level, continent, prevailing wage, number of employees, and visa status.

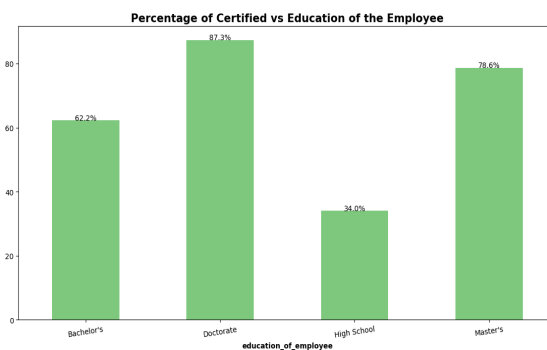
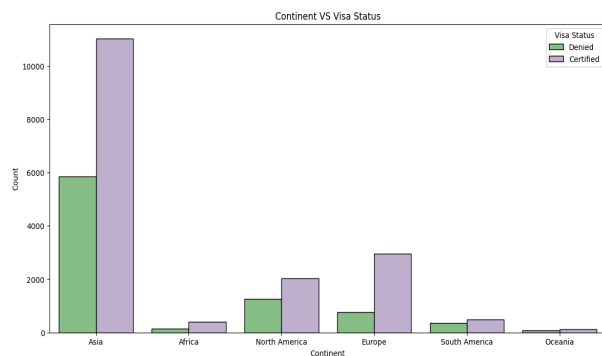
Distribution of Numerical Features By Case Status



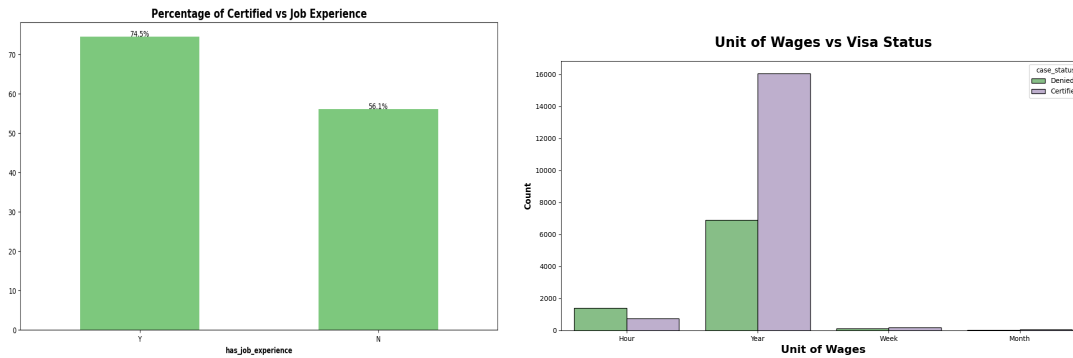
High Level Design (HLD)



Multivariate analysis was conducted to study the relationships between multiple features and their influence on visa approval outcomes. Comparative visualizations and statistical analyses were used to examine how factors such as education level, work experience, prevailing wage, company size, region of employment, and full-time position status affected visa certification rates.



High Level Design (HLD)



3.1.3 Data Validation

The Data Validation component ensures that the ingested dataset conforms to the expected schema and quality requirements. Validation checks are performed to verify the presence of required columns, data consistency, and structural correctness. Additionally, data drift detection is implemented using Evidently AI to compare the distribution of incoming data with the training dataset. Validation reports are generated automatically to identify potential issues before model training.

3.1.4 Data Preprocessing

Data preprocessing was performed to improve the quality and consistency of the visa application dataset before model training. Missing numerical values were handled using mean imputation, while missing categorical values were replaced using mode imputation. Duplicate records were identified and removed to eliminate redundancy and ensure data integrity. The target variable, `case_status`, was converted into numerical format to make it suitable for machine learning algorithms. These preprocessing steps ensured that the dataset was clean, consistent, and ready for further analysis and model development.

```

1 columns:
2   - case_id: category
3   - continent: category
4   - education_of_employee: category
5   - has_job_experience: category
6   - requires_job_training: category
7   - no_of_employees: int
8   - yr_of_estab: int
9   - region_of_employment: category
10  - prevailing_wage: int
11  - unit_of_wage: category
12  - full_time_position: category
13  - case_status: category
14  - company_age: int
15
16 numerical_columns:
17   - no_of_employees
18   - yr_of_estab
19   - prevailing_wage
20
21 categorical_columns:
22   - case_id
23   - continent
24   - education_of_employee
25   - has_job_experience
26   - requires_job_training
27   - region_of_employment
28   - unit_of_wage
29   - full_time_position

```

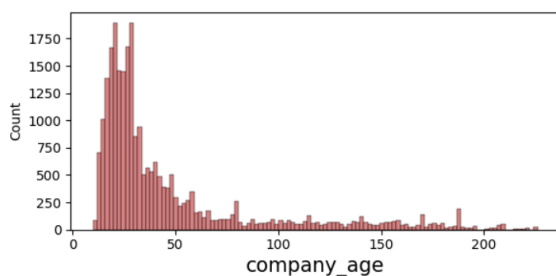
3.1.5 Feature Engineering

Feature engineering was performed to create meaningful features that could improve the predictive performance of the machine learning models. Based on insights obtained from Exploratory Data Analysis (EDA), a new feature called Company Age was created by calculating the difference between the current year and the company's year of establishment. This derived feature provided additional information about the employer's background and stability, which could influence visa approval decisions. The engineered feature helped enhance the dataset's predictive capability and contributed to improved model performance.

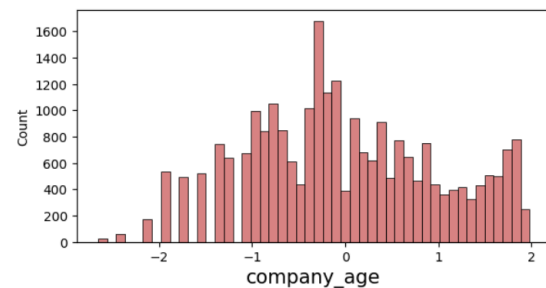
3.1.6 Data Transformation

Data transformation was performed to convert the preprocessed data into a format suitable for machine learning algorithms. Categorical features were encoded using One-Hot Encoding and Ordinal Encoding, while numerical features were standardized using StandardScaler to ensure consistency across different scales. Additionally, highly skewed features such as Company Age and Number of Employees were transformed using the Yeo-Johnson Power Transformation to achieve a more normal distribution and improve model learning. These transformations were implemented through an automated preprocessing pipeline using ColumnTransformer and Pipeline, ensuring consistent data processing during both training and prediction.

Before Power Transformation

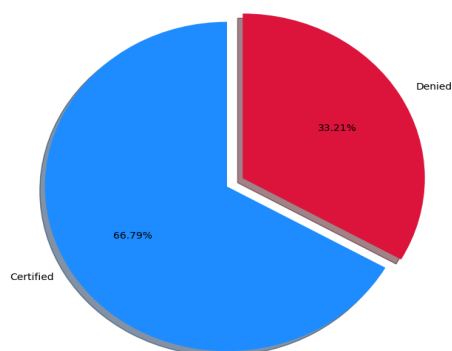


After Power Transformation



3.1.7 Class Imbalance Handling

The visa application dataset contained an imbalance between approved and denied visa cases, which could lead to biased model predictions.

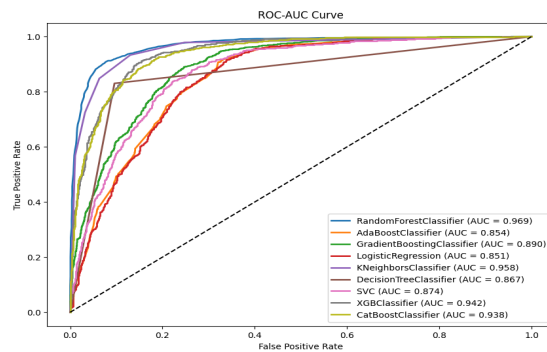


To address this issue, the SMOTEENN technique was applied during the data transformation stage. SMOTE (Synthetic Minority Oversampling Technique) generated synthetic samples for the minority class to improve class representation, while Edited Nearest Neighbors (ENN) removed noisy and misclassified observations from the dataset. This combined approach helped create a more balanced dataset, improved the model's ability to learn from both classes, and enhanced overall prediction performance.

3.1.8 Model Training

Multiple machine learning classification algorithms were trained and evaluated to identify the most suitable model for visa approval prediction. The models used included Random Forest Classifier, AdaBoost Classifier, Gradient Boosting Classifier, Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree Classifier, Support Vector Classifier (SVC), XGBoost Classifier, and CatBoost Classifier. The performance of all models was compared using evaluation metrics, and the results were summarized in a comparison table. The models were evaluated using the ROC-AUC metric. ROC-AUC was selected because it measures the model's ability to distinguish between approved and denied visa applications across different classification thresholds.

	Model Name	Train Accuracy	Test Accuracy	F1 Score	Precision	Recall	ROC AUC
0	RandomForestClassifier	1.000000	0.916395	0.902160	0.901015	0.903308	0.968872
4	KNeighborsClassifier	0.943801	0.899023	0.877578	0.909091	0.848176	0.957656
7	XGBClassifier	0.946787	0.871516	0.853729	0.830128	0.878711	0.942001
8	CatBoostClassifier	0.903077	0.863916	0.845649	0.819411	0.873622	0.938191
2	GradientBoostingClassifier	0.814208	0.807094	0.783245	0.752344	0.816794	0.890289
6	SVC	0.794118	0.796960	0.763191	0.759664	0.766751	0.873918
5	DecisionTreeClassifier	1.000000	0.872602	0.847619	0.865606	0.830365	0.867203
1	AdaBoostClassifier	0.758914	0.757148	0.702703	0.735622	0.672604	0.853973
3	LogisticRegression	0.761357	0.756786	0.708333	0.725333	0.692112	0.851458

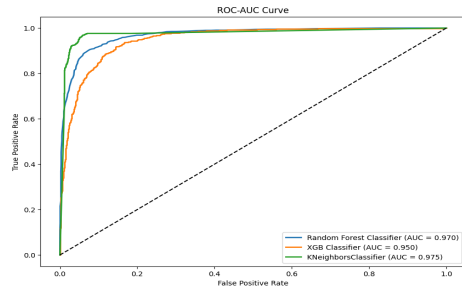


Based on the evaluation results, the Random Forest Classifier achieved the highest ROC-AUC score and was selected as the final model.

3.1.9 Hyperparameter Optimization

To further enhance model performance, GridSearchCV was applied to the top-performing models: Random Forest, XGBoost, and KNN. A 3-fold cross-validation strategy was used, where the training dataset was divided into four subsets and evaluated across multiple iterations. Various combinations of hyperparameters were tested systematically to identify the optimal settings for each model. This process helped improve predictive performance, reduce overfitting, and ensure better generalization on unseen data. The optimized models were then evaluated using the ROC-AUC score to determine the best-performing model for deployment.

High Level Design (HLD)



	Model Name	Train Accuracy	Test Accuracy	F1 Score	Precision	Recall	ROC AUC
2	KNeighborsClassifier	1.000000	0.945349	0.934147	0.961400	0.908397	0.940625
0	Random Forest Classifier	1.000000	0.913500	0.898857	0.896959	0.900763	0.911872
1	XGB Classifier	0.970407	0.884184	0.867330	0.848337	0.887193	0.884568

The tuning process resulted in significant performance improvements, with KNN and Random Forest emerging as the best-performing models, achieving the highest ROC-AUC scores among all models tested. These optimized models were then considered for final evaluation and deployment.

3.1.10 Model Evaluation

The model evaluation process involved loading the test dataset, applying necessary feature engineering, and evaluating the newly trained model against the existing production model retrieved from AWS S3. Both models were compared using F1-score as the primary evaluation metric. The new model was accepted only if it showed improved performance over the production model; otherwise, the existing model was retained. Finally, a model evaluation artifact was generated containing the acceptance decision, performance difference, and relevant model paths to support deployment.

3.1.11 Model Pusher

The model pusher component is responsible for deploying the finalized trained model to AWS S3 after successful evaluation. Once the model is accepted, it is automatically uploaded to cloud storage using a dedicated pipeline, ensuring a smooth and automated deployment process. The trained model artifact is saved in a centralized S3 bucket, enabling version control, scalability, and easy access for production use. This integration with AWS S3 ensures that the model is readily available for API deployment and real-time inference, supporting a reliable and maintainable machine learning workflow.

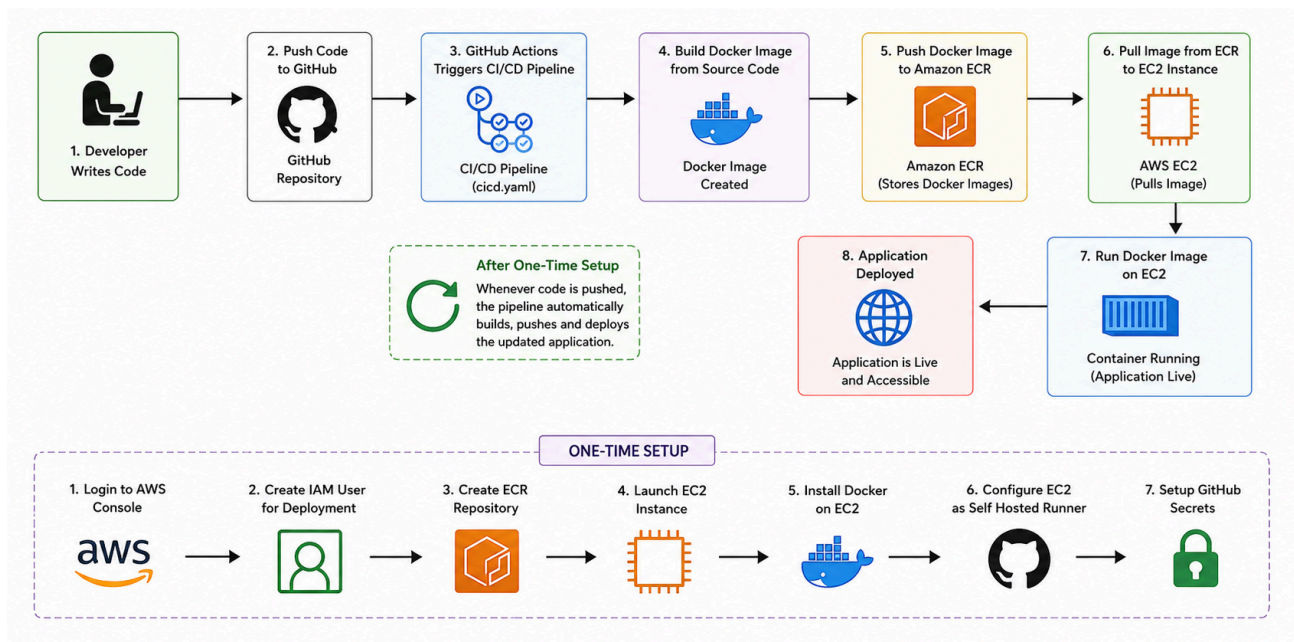
3.1.12 Prediction Pipeline

The prediction pipeline is responsible for handling real-time inference for the US visa approval system. It takes user input features such as applicant details and converts them into a structured DataFrame using the VisaData class. This input is then passed to the VisaClassifier, which loads the trained model from AWS S3 using the VisaEstimator. The model processes the input data and generates predictions for visa approval status. This pipeline ensures seamless integration between user input and the deployed machine learning model, enabling fast, scalable, and real-time predictions in production.

3.1.13 FastAPI Web Application

The application is built using FastAPI to serve as the deployment layer for the US Visa Approval prediction system. It provides endpoints for both model training and real-time prediction. The /train endpoint triggers the training pipeline, allowing the model to be retrained when required. For inference, user inputs are collected through an HTML form, structured using the VisaData class, and passed to the VisaClassifier, which loads the trained model from AWS S3 and generates predictions. The application uses Jinja2 templates to render the frontend and display results as either “Visa Approved” or “Visa Not Approved.” This architecture ensures efficient integration of the machine learning pipeline with a lightweight and scalable web framework for real-time deployment.

3.1.14 Deployment Process



The deployment process begins when the developer pushes the application code to the GitHub repository. This automatically triggers the CI/CD pipeline through GitHub Actions, which builds a Docker image from the latest source code. The generated Docker image is then pushed to Amazon Elastic Container Registry (ECR), which serves as a centralized storage location for Docker images. Next, the AWS EC2 instance pulls the latest image from ECR and runs it as a Docker container. Once the container starts successfully, the application becomes live and accessible to users. After the initial setup, which includes creating an IAM user, configuring ECR and EC2,

installing Docker, setting up the EC2 instance as a self-hosted runner, and configuring GitHub secrets, the entire deployment process becomes fully automated. As a result, whenever new code is pushed to GitHub, the application is automatically built, deployed, and updated without requiring any manual intervention, ensuring faster and more reliable software delivery.

3.2 Event log

- Tracks important activities throughout the US Visa Approval Prediction pipeline.
- Records events during data ingestion, data validation, transformation, model training, evaluation, and prediction.
- Monitors model comparison, model pushing, and deployment workflows.
- Stores timestamps and execution details for effective debugging and auditing.

- Helps identify failures and bottlenecks during pipeline execution.
- Improves traceability, maintainability, and monitoring of the application.

3.3 Error Handling

- Detects and handles exceptions across all stages of the machine learning pipeline.
- Prevents application crashes using custom exception handling mechanisms.
- Generates meaningful error messages to simplify debugging and troubleshooting.
- Logs errors and failure points for monitoring and maintenance.
- Ensures smooth execution of training, evaluation, deployment, and prediction processes.
- Enhances the reliability, robustness, and stability of the system.

4 Performance

- Achieves reliable visa approval predictions through machine learning-based classification.
- Evaluates model performance using metrics such as Accuracy, Precision, Recall, and F1-Score.
- Hyperparameter tuning improves model effectiveness and prediction accuracy.
- K-Nearest Neighbors (KNN) achieved the best performance among the evaluated models.
- Supports real-time prediction with low response time through the FastAPI application.
- Automated model evaluation ensures only higher-performing models are deployed to production.
- Consistently processes unseen data to generate accurate and reliable predictions.
- Cloud-based model storage enables efficient model retrieval and inference.
- End-to-end pipeline automation reduces processing time and improves operational efficiency.

5 Conclusion

In conclusion, the US Visa Approval Prediction project successfully demonstrates the application of machine learning to automate and enhance the visa decision-making process. The system follows a complete end-to-end pipeline, starting from data ingestion from MongoDB, followed by preprocessing, feature engineering, and exploratory data analysis. Multiple classification models were trained and evaluated using performance metrics such as F1-score to ensure reliable predictions. The best-performing model was selected, tuned for optimal performance, and integrated with a robust deployment pipeline. The final model is stored in AWS S3 and served through a FastAPI-based web application, enabling real-time predictions. This ensures scalability, maintainability, and efficient access for end users. Overall, the project delivers a production-ready, automated, and data-driven solution for predicting visa approval outcomes with improved accuracy and reliability.

6 References

Historical Immigration / Visa Application Dataset — Kaggle
Research Papers and Online Resources related to:

- Machine Learning
- MLOps
- Visa Approval Prediction Systems
- Data Drift Monitoring
- Cloud Deployment and CI/CD Pipelines